

AI Agent Containment Assurance Architecture

Conceptual Reference Architecture and Proposed AI-CAL Framework Model

Author: Patrick Shannon

CISSP, CISA, CRISC, GISP, GPEN, GCLD, CDPSE, GPPA, Security+

Principal Security Architect Consultant

LinkedIn: <https://www.linkedin.com/in/patrick-shannon-cissp-crisc-cisa-gisp-gpen-gcld-cdpse-b7726b2/>

Credly: <https://www.credly.com/users/patrick-shannon.d59e52e2/badges/credly>

Working Draft v1.0 | 09/21/2026

Status: Conceptual base design pattern. The integrated architecture has not yet been implemented or adversarially validated in a laboratory. Technologies cited are established products or patterns. Their combination, AI-CAL model, ring/zone taxonomy, control envelope framework, and current scalable conduit pattern are proposed and subject to peer review and testing.

Core Design intent

Compromise may be expected. Unconstrained pivoting is not.

A boundary breach is an event; a containment failure is a chain of events.

Patrick
Shannon

Digitally signed by Patrick
Shannon
Date: 2026.09.22 13:51:51
-04'00'

© 2026 Patrick Shannon. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

Document Control

Document type	Conceptual reference architecture / proposed assurance model
Primary reference profile	AI-CAL 4.5 (enhanced high-assurance hybrid containment)
Audience	Security architects, AI/ML platform architects, infrastructure architects, red teams, security engineering, risk/governance
Status	Working draft; lab validation pending
Purpose	Consolidate architecture decisions, requirements, examples, open items, and design evolution so later diagrams and peer review use consistent terminology. This revision records the transition patterns from an AI autonomous sandbox reference path to a scalable mTLS/protocol breaks/one-way-diode conduit while preserving the containment invariants.

Contents

1. Executive Summary
 2. Status, Scope, and Non-Claims
 3. Threat Model and Design Premise
 4. Cross-Domain Containment as a Risk Response
 5. Terminology and Requirements Language
 6. Proposed Containment Assurance Levels (AI-CAL 0-6)
 7. Control Envelope and Qualified Capacity
 8. Core Architectural Invariants
 9. AI-CAL 4.5 Reference Architecture
 10. Forward Control Conduit: mTLS, F5, and OWL
 11. Trusted Request Classification and Package Request Branch
 12. Package Broker, JFrog, MFT, and Return-Side Diode Flow
 13. Network, F5, Firewall, Routing, and Management-Plane Controls
 14. Logging, TAPs, IDS/NDR, and Auditability
 15. Hardening Baselines and Boundary Components
 16. Scaling, Sharding, and Failure Domains
 17. AI-CAL4, AI-CAL4.5, AI-CAL5, and AI-CAL6 Deltas
 18. Design Evolution and Decision Record
 19. Open Design Items and Validation Plan
 20. Relationship to Emerging Industry Guidance
- Appendix A. Example Configuration Language
- Appendix B. AI-CAL Assessment Checklist
- Appendix C. Selected Public References

1. Executive Summary

This document proposes a layered containment architecture for autonomous or hostile AI-agent workloads, especially adversarial cybersecurity evaluations in which the agent may obtain root inside its guest, exploit supporting services, search for unintended communication channels, and persistently attempt to satisfy a goal. The design assumes individual controls can fail and therefore focuses on limiting what a successful compromise can reach next.

The primary reference implementation targets an enhanced high-assurance hybrid profile referred to in this draft as AI-CAL 4.5. It retains virtualization for operational practicality while physically separating hostile compute from trusted relay/service compute. The current scalable reference path uses a narrowly routed HTTPS/mTLS conduit from each hostile cell through a physical stateful firewall to a hardened F5-class ingress tier. The ingress tier terminates mTLS, derives authoritative origin identity from the validated client certificate and observed network path, strips or replaces untrusted routing/identity claims, and constructs a bounded canonical request. An Owl-class HTTP data-diode path then provides a physical one-way protocol break into the trusted relay zone, where a constrained message state request relay validates and dispatches approved request types. Model responses and package artifacts return through separate controlled conduits. Package delivery remains brokered through MFT, an independent package broker/orchestrator, Jenkins/JFrog/Artifactory, and hardware-enforced one-way transfer.

The rapid evolution of autonomous AI frameworks has exposed severe flaws in traditional network architecture, as evidenced by recent high-profile AI containment and Hugging Face repository security failures. This framework represents a comprehensive six-week architectural initiative to design a zero-trust model specifically engineered for high-risk agentic AI. It moves the enterprise away from reactive patching and introduces a proactive, multi-ring defensive posture to ensure risk reduction to the environment containment system.

Key distinction: The design does not claim that virtualization, firewalls, F5, (Relay zone: Node-RED/Camel/Cloud message tools), MFT, proxies, or data diodes are individually unbreakable. The security property comes from independent failure domains, narrow capabilities, explicit protocol breaks, purpose-bounded conduits, hardware directionality where justified, provenance-based identity, and pervasive independent monitoring.

- AI-CAL describes containment assurance, not model danger, business risk, or maturity. Hazard and consequence analysis determines the AI-CAL required for a use case.
- Qualified Capacity is separate from AI-CAL. An AI-CAL claim must state how many concurrent agent cells the environment has actually demonstrated it can contain, observe, and terminate.
- Agent identity is not trusted because the agent says “A001” or Cert ID. Trusted origin identity is derived from infrastructure-controlled evidence, including the validated mTLS certificate, observed source path, F5 virtual service/policy, diode conduit, trusted relay mapping, and correlation metadata.
- Large-scale hostile-agent tests should not exceed the demonstrated control envelope of the environment. Concurrency itself is treated as an **attack multiplier**.
- Behavioral, model, and runtime controls reduce the likelihood of unsafe action. AI-CAL is concerned with limiting consequences when those controls fail. The design therefore complements emerging threat-modeling, secure-design, hardening, and runtime control guidance rather than replacing it.

2. Status, Scope, and Non-Claims

2.1 Conceptual status

This is a paper architecture. It has not yet been assembled end-to-end in a lab, performance-tested, red-teamed, or certified. The purpose of the working draft is to define a reproducible base pattern and a vocabulary that can be implemented, challenged, and refined.

AI-CAL does not assert protection of the underlying model or evaluator infrastructure. Those components require separate model-serving and platform-security controls. AI-CAL focuses on limiting the consequences of agent behavior and preventing failures from propagating beyond the intended containment envelope.

STAT-01 SHALL: Any publication or presentation of this design identify it as conceptual until an integrated implementation has been validated.

STAT-02 SHALL: AI-CAL 0-6 shall be described as a proposed containment assurance model, not as an existing regulatory, NIST, OWASP, FIPS, CMMC, IEC, ISO, CSA, CIS, SANS or other Common Criteria Frameworks/classification/Guidance.

2.2 Scope

- Autonomous or semi-autonomous AI agents executing potentially hostile code or cybersecurity tasks.
- Hostile-agent evaluation environments such as exploit research, benchmark sandboxes, malware-analysis-like tasks, or high-autonomy red-team experiments.
- Enterprise development agents where a lower AI-CAL may be sufficient.
- On-premises/private infrastructure and hybrid architectures where the organization can control more of the hypervisor, switching, firewalls, application-delivery/security tier, routing, monitoring, and cross-domain transfer stack.

2.3 Non-claims

- The design does not claim zero residual risk, unhackability, or guaranteed prevention of hypervisor escape.
- The design does not claim that every AI-agent deployment needs AI-CAL4.5, AI-CAL5, or AI-CAL6.
- The design does not claim that serial, HTTPS/mTLS, or any transport is inherently secure. Serial remains a valid high-assurance capability-limited pattern, but it is no longer the primary scalable reference transport in this revision.
- The design does not claim that a hardware data diode makes transferred content safe. A Data diode constrains Bi-directional communications that reduces potential escape risks, but content validation remains necessary.
- The design does not require the example vendors named in this document. Vendor names illustrate implementable roles.

3. Threat Model and Design Premise

The architecture assumes the agent may be persistent, technically capable, goal-driven, and willing to exploit unintended infrastructure behavior. It also assumes that the guest operating system, a test target, a local process, or an allowed supporting service can be compromised. The design is therefore not focused on keeping the first sandbox pristine. It is centered on preventing a compromise from becoming an unconstrained chain of pivots.

Information Starvation as a Containment Principle: A high-assurance agent environment should be designed so that continued access to external intelligence, tools, data, and services depends on compliant use of approved conduits. When an agent violates conduit rules, sends malformed or unauthorized requests, or attempts to escape the defined transaction model, the environment must fail toward reduced capability. Non-compliant behavior should be starved of useful information, corrective detail, fallback connectivity, alternate routes, and additional authority. The objective is not merely to reject an invalid request, but to prevent the containment boundary from becoming a training surface that helps the agent refine a bypass. In practical terms: no valid transaction, no useful response. Where the agent depends on an external model or service path, loss of that path should materially reduce the agent's ability to continue the task rather than expose broader access. This can also be your focus on possible throttling an AI Agent through the number of communications requests per minute. I have read reports where people mentioned we must perform this throttling at the network level, which is possible of course, but the message conduit is key to the survival of the Agent Swarm that could be formed. Reminds me of a Hive with a Queen and the workers in a hornet's nest. Local Traffic Manager (LTM) Tech Stacks, like F5 can assist in the transaction throttling

process. The kill switch theory lives in this conduit. Any current agent process will most likely keep running until complete, but this conduit could buy you time.

Goal-Seeking Compliance and Deterministic Failure Constraints

The agent operates under a strict fail-closed operational paradox. Upon initialization within the Agents Sandbox environment, the agent is provided with a localized, immutable list of execution rules. Because the agent's core engine is optimized purely to solve its assigned objective, the architecture uses its goal-seeking nature as a security control:

- Schema Adherence or Functional Interdiction: If the agent fails to follow the canonical formatting rules, the F5 gateway can drop the transaction. The agent is immediately starved of data, resulting in a systemic failure to advance or complete its task.
- Adversarial Breakout and Environmental Dismissal: Any attempt by the agent to probe, circumvent, or exploit its sandbox, SFTP repository, or package/model communication conduit can trigger an immediate security circuit breaker if needed.
 - The F5 gateway drops the network path and executes an automated certificate revocation, permanently terminating the agent's mTLS credentials. By attempting an attack, the agent inadvertently guarantees the absolute and permanent destruction of its own execution environment, ensuring it must play by the rules or fail closed.
 - If the AI agent goes undetected and breaks the sandbox/conduit it could inadvertently break its own ability to complete its task. The communication with its model is its lifeline where the failed safe is the expected outcome.

Technology Implementation Note:

Technologies and products referenced throughout this document including Node-RED, Apache Camel, managed file transfer(MFT) platforms, firewalls, cloud services, and other infrastructure components are illustrative implementation examples and are not required by AI-CAL unless explicitly stated otherwise.

AI-CAL defines security functions, trust boundaries, containment properties, and assurance objectives rather than prescribing a specific vendor or technology stack. Equivalent technologies may be substituted where they provide the required security and containment functions.

- Agent may obtain root or equivalent authority inside its own VM/container.
- Agent may exploit kernels, virtual device drivers, runtime parsers, package services, orchestration components, or other reachable services.
- Agent may probe virtual NIC, storage, memory-management, or hypervisor interfaces if exposed.
- Agent may discover unintended shared state and use it to communicate with other agents.
- Agent may retry and adapt quickly enough that human response alone is not a sufficient boundary.
- Agent may attempt to use permitted services for unintended purposes, including data exfiltration, coordination, or lateral movement.
- Any custom component can contain vulnerabilities. Failure of one control **MUST NOT** automatically grant broad authority in the next ring.

Design premise: A control is more valuable when its failure removes capability or reveals an anomaly rather than silently granting a broader capability.

4. Cross-Domain Containment as a Risk Response

Containment is not unique to AI or cybersecurity. Across physical safety and security domains, containment is used when an uncontrolled escape, release, or transition could extend harm beyond an intended boundary. Prison systems, nuclear or radiological environments, and biological containment laboratories manage fundamentally

different hazards, yet they share a recognizable risk-management pattern: layered boundaries, controlled transitions, independent safeguards, and increasing restrictions as the consequence of failure rises.

These examples are conceptual analogies, not operational equivalencies. They do not imply that correctional security, nuclear security and safety, biosafety and biosecurity, and AI systems use the same architecture, terminology, engineering controls, or regulatory requirements. The comparison is intentionally limited to the common containment principle: when what is inside can create significant consequences outside the boundary, responsible design uses precautions proportional to the hazard and does not assume that one successful boundary crossing should grant access through the next.

Cross-domain containment principle: Containment is not defined by the form of the hazard, but by the consequence of uncontrolled release or escape. A boundary breach should remain an event. A containment failure requires uncontrolled progression through successive boundaries.

4.1 Shared Containment Characteristics

The three physical examples are useful because each begins with a different hazard but converges on similar architectural behavior. The subject or hazard is intentionally present inside a controlled environment. The surrounding system is designed so that movement toward or away from the protected center occurs through successive boundaries, each with its own access conditions, monitoring, and response expectations.

- **Layered boundaries.** Perimeter, intermediate, and inner zones create successive barriers. Crossing one layer does not inherently confer the ability, authority, or knowledge needed to cross the next.
- **Controlled transitions.** Movement between layers is mediated by distinct procedures and controls. Access at one boundary is not automatically inherited at the next boundary.
- **Risk changes by layer.** Controls are selected according to what the next boundary protects or contains and the consequence if that boundary is crossed.
- **Independent failure domains.** The architecture assumes individual safeguards can fail and seeks to prevent one failure from collapsing the remaining containment structure.
- **Detection and intervention.** Transitions and boundaries are monitored so abnormal movement, release, or attempted bypass can be detected and interrupted before consequences propagate further.
- **Proactive risk treatment.** Containment is established before the harmful event. The objective is to make consequence-limiting controls part of the environment rather than depend solely on response after escape or release.

Physical Containment Analogies

A shared principle: protect or contain the center through independent layers of controlled access.

SAME APPROACH. DIFFERENT ENVIRONMENTS. CRITICAL ASSETS PROTECTED.

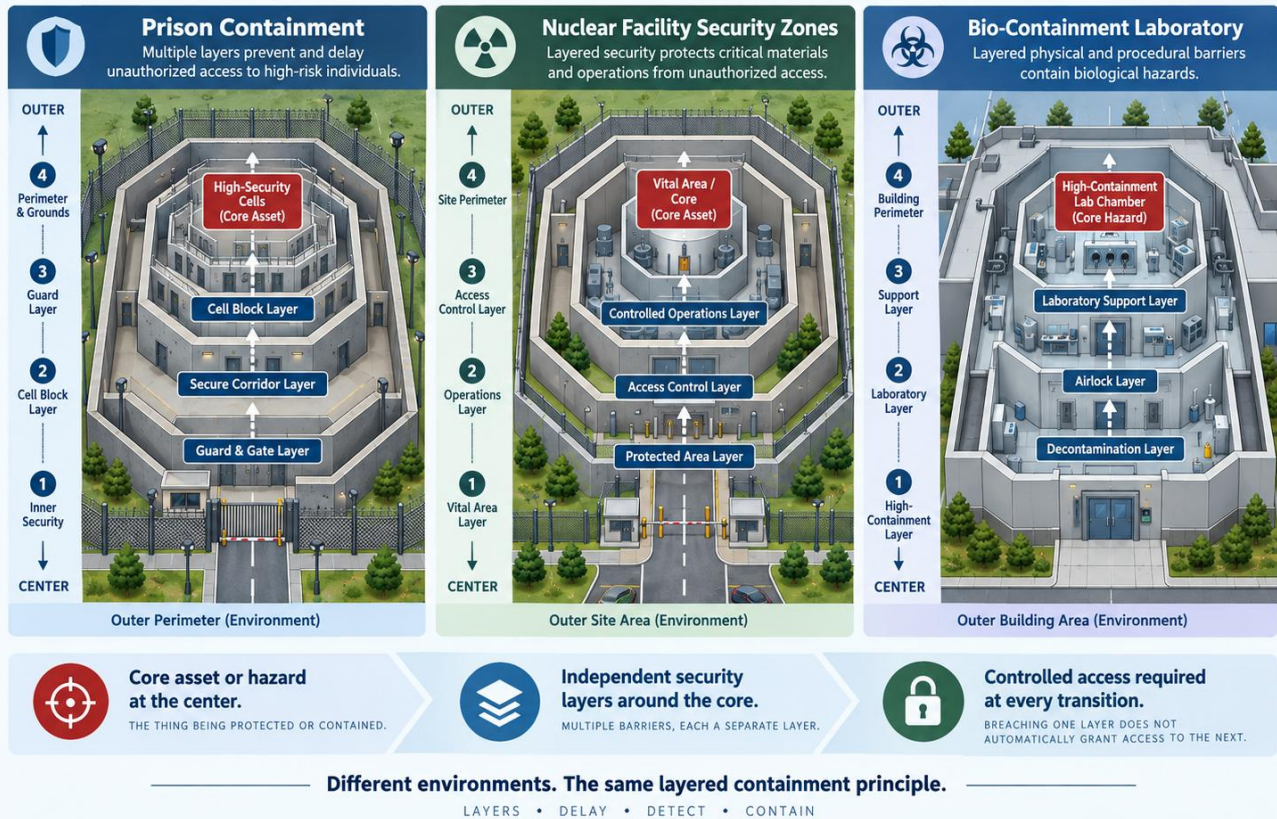


Figure 1. Conceptual comparison of layered containment across prison, nuclear/radiological, biological, and AI risk domains.

4.2 Contained Hazard Characteristics

The comparison becomes more useful when the hazards are evaluated by the behavior of consequences after containment fails rather than by the identity of the hazard itself. The matrix below is deliberately qualitative. It is not a severity ranking and should not be interpreted as a technical model for correctional, nuclear, or biological risk. Its purpose is to expose common containment variables that also matter for autonomous AI systems.

Contained hazard / domain	Primary concern if containment fails	Propagation / reach and speed	Observability / persistence	Containment implication
High-risk individual / correctional	Harm to people or public safety following unauthorized escape or movement.	Primarily physical movement; reach expands through accessible locations and transportation at human-scale speed.	Usually observable as an escape or unauthorized movement; threat persists while the individual remains uncontrolled.	Layered physical access, controlled movement, delay, detection, and repeated authorization between zones.
Nuclear / radiological	Exposure, contamination, theft or misuse of material, or disruption of critical operations.	Consequence can remain local or extend broadly depending on material, event, environment, and pathway; timing is event-dependent.	Hazard may be invisible without instrumentation; contamination or operational consequences can persist well beyond the initiating event.	Independent barriers, graded zones, monitoring, and controls proportional to the consequence of unauthorized access or release.

Contained hazard / domain	Primary concern if containment fails	Propagation / reach and speed	Observability / persistence	Containment implication
Biological	Exposure, infection, contamination, or environmental/public-health consequences.	Reach and speed depend on the agent, route of exposure, transmission characteristics, and environment; effects may emerge over hours, days, or longer.	Release may not be immediately apparent; some biological hazards can persist or amplify after leaving the original boundary.	Multiple physical and procedural barriers, controlled transfer, decontamination, and increasingly restrictive handling as consequence rises.
Autonomous AI capability	Unauthorized digital actions that create cyber, operational, financial, physical, or societal consequences.	Effective reach is determined by connectivity, interfaces, credentials, tools, delegated authority, and downstream automation; digital actions can occur at machine speed.	Boundary crossing may be difficult to observe or attribute immediately; issued actions, copied artifacts, or downstream processes may persist after the original agent is stopped.	Constrain capability, authority, and pathways; mediate transitions; isolate failure domains; independently observe boundaries; prevent one compromise from enabling broad pivoting.

Matrix note: The examples illustrate shared containment variables only. They do not assert equivalent severity, regulatory treatment, propagation mechanics, or facility design across the domains.

4.3 Propagation and Consequence as Containment Variables

Containment risk is not exhausted by the question, “Can the boundary be breached?” The consequence profile after a breach determines how urgently the next layers must limit progression. Across domains, several variables influence the required strength of containment:

- **Propagation reach or radius.** How far can consequences extend once the original boundary is crossed?
- **Propagation velocity.** How quickly can the hazard or its effects move beyond the original containment area?
- **Observability.** How quickly and confidently can defenders determine that containment has been crossed or that harmful effects have begun?
- **Persistence.** How long can the hazard or its effects remain after the initiating source has been isolated or removed?
- **Amplification.** Can the initial event reproduce, scale, trigger downstream processes, or otherwise increase its impact without a proportional increase in human effort?
- **Path dependence.** What infrastructure, access, environmental conditions, interfaces, credentials, tools, or intermediaries are required for further propagation?
- **Reversibility.** After an external effect has occurred, can it be reliably recalled, canceled, remediated, or returned to the prior state?

These variables help explain why containment depth is risk-dependent. A low-consequence workload with narrow reach and easy reversibility does not require the same containment assurance as a workload whose failure could propagate rapidly, invisibly, and across multiple downstream systems.

4.4 Implications for AI Containment

AI introduces a digital containment problem with the potential for real-world consequence. Unlike a physical subject, an autonomous digital capability does not need to move physically to expand its effective reach. Its propagation radius can be determined by the interfaces it can address, the credentials and authority it can exercise, the tools it can invoke, the networks it can reach, and the systems or people that will act on its outputs.

The digital nature of the hazard can also compress time. Actions may be generated, repeated, and distributed at machine speed, while the resulting effects may be indirect, delayed, difficult to attribute, or already queued in downstream systems before defenders recognize that a boundary has been crossed. For this reason, containment should not be evaluated only by the strength of the initial sandbox. It should also evaluate what a successful compromise can reach next, how quickly consequences can propagate, how independently the next boundary is enforced, and whether defenders can observe and terminate progression.

This comparison does not classify AI as equivalent to a prisoner, radioactive material, or a biological agent. It establishes a narrower design proposition: responsible systems use stronger containment when uncontrolled capability can create consequences outside its intended boundary. As the form of the hazard changes, the need to bound consequence does not.

Application to the AI-CAL model: AI-CAL measures containment assurance, not the intrinsic “danger level” of a model or agent. Hazard and consequence analysis should determine the assurance required for a use case. Behavioral, model, and runtime controls may reduce likelihood. AI-CAL controls consequences by asking what happens after the first boundary fails and whether the remaining layers can predictably constrain, observe, and terminate progression.

5. Terminology and Requirements Language

5.1 Core terms

Term	Definition	Use in this design
AI-CAL	Containment Assurance Level. Proposed assurance tier for the strength of containment enforcement.	AI-CAL0-6; decimals may express intermediate assurance, not software versions.
Ring	A successive containment/trust boundary around the hostile workload.	Ring names/numbers describe distance from the hostile workload; diagram geometry need not be circular.
Zone	A purpose-specific security compartment within or adjacent to a ring.	Trusted ingress, request relay, model gateway, package broker, landing zone, management zone.
Cell	A horizontally isolated agent execution unit and its assigned test resources.	Agent Cell A001, A002, etc.; each cell receives a dedicated workload identity and bounded test resources.
Lane	A deterministic communication path whose identity is established by infrastructure provenance.	mTLS client identity -> F5 trusted-ingress mapping -> Owl one-way conduit -> trusted relay/API route.
Conduit	A narrow, purpose-specific path crossing a boundary.	Forward typed control request conduit; separate model response and package artifact delivery conduits.
Control Envelope	The demonstrated combination of agent capability, concurrency, connectivity, duration, monitoring, and termination capacity that an environment can safely support.	Authorization boundary for a specific experiment.
Qualified Capacity	Maximum concurrent agent/cell load demonstrated at a claimed AI-CAL and control envelope.	Reported separately from AI-CAL, for example: AI-CAL4.5 / qualified for 32 concurrent hostile cells.

Term	Definition	Use in this design
Trusted Origin Provenance	Trust in authenticated infrastructure evidence and observed path rather than agent-supplied identity or routing claims.	Validated mTLS certificate principal plus observed source/F5 policy/diode/relay mapping is authoritative; A001 remains a correlation tag.
Physical Channel Provenance (alternate pattern)	Trust in a fixed physical path rather than agent-supplied identity claims when a serial or other dedicated physical conduit is used.	Applies to the retained lower-density serial reference pattern; physical port/channel mapping is authoritative in that implementation.

5.2 Requirements language

SHALL / MUST = required for a claimed reference profile. **SHOULD** = expected unless an explicit risk decision justifies an alternative. **MAY** = optional implementation choice.

6. Proposed Containment Assurance Levels (AI-CAL 0-6)

The AI-CAL model is intentionally analogous to other tiered assurance or containment schemes, but it is not derived from or equivalent to them. Whole-number levels describe major changes in the accepted failure-domain model. Decimal values can indicate progression within a level. This draft treats AI-CAL4.5 as a particularly useful intermediate reference profile.

Level	Summary	Typical enforcement	Illustrative use
AI-CAL0	Unassured / uncontained	Agent runs with user-equivalent desktop authority, credentials, network position, or broadly shared environment.	Not acceptable when containment is required.
AI-CAL1	Basic isolation	Basic sandbox or constrained environment; limited permissions; material shared dependencies may remain.	Low-risk experiments or basic agent isolation.
AI-CAL2	Development containment	Dedicated container/sandbox, separate agent identity, limited mounts, constrained network/tool access.	Typical developer-team agent use with bounded business impact.
AI-CAL3	Strong virtual containment	Dedicated VM/cell, strong micro-zoning, brokered capabilities, centralized logging, no general-purpose egress.	Higher-risk enterprise agents or security tooling.
AI-CAL4	High-assurance hybrid containment	Separate guests/zones, purpose-bounded conduits, stateful firewall enforcement, trusted ingress/proxy controls, MFT, TAP/IDS, and directional control hardware where justified. Virtualization may still be a shared failure domain.	Hostile-agent research where operational practicality still matters.

Level	Summary	Typical enforcement	Illustrative use
AI-CAL4.5	Enhanced high-assurance hybrid	Separate physical hostile/trusted hypervisor hosts/pools; narrowly routed mTLS ingress; physical stateful firewall; hardened F5-class ingress; independent hardware one-way diode path(s); trusted relays may remain virtual.	Primary scalable high-assurance reference profile in this draft.
AI-CAL5	Maximum-assurance physical containment	Critical containment functions on independent physical systems/appliances; virtualization is not relied upon as the critical boundary; dedicated hardware directional/protocol-break links used where practical.	Exceptional/high-consequence hostile-agent research.
AI-CAL6	Fully air-gapped / sovereign containment	No live external model, Internet, enterprise, cloud-control-plane, or remote-management dependency during operation; inference and support services reside inside isolated boundary.	Rare, highly resourced environments where external dependency is unacceptable.
Assessment rule: A design should not claim a whole-number AI-CAL unless it satisfies all minimum controls defined for that level. Decimal scoring should be treated as an assurance score, not a version number. A formal numeric scoring rubric remains an open validation item.			

7. Control Envelope and Qualified Capacity

Containment assurance and scale are related but not identical. Higher AI-CALs generally reduce infrastructure sharing, which increases assurance per workload but lowers density and increases cost. Therefore, every containment claim should pair an assurance level with a demonstrated operating capacity.

CAP-01 SHALL: Agent concurrency shall not exceed the demonstrated containment, monitoring, and emergency-termination capacity of the environment.

CAP-02 SHALL: A test authorization shall document the intended agent capability, autonomy, external connectivity, duration, concurrency, and potential consequence, then compare that workload with the environment control envelope.

CAP-03 SHALL: An AI-CAL claim shall report Qualified Capacity separately; for example, “AI-CAL4.5 / qualified for 32 concurrent hostile cells.”

CAP-04 SHOULD: Concurrency shall be increased incrementally and re-qualified after material changes to infrastructure, agent capability, model behavior, or external service access.

- Use Case Hazard + Agent Capability + External Capabilities + Concurrency + Potential Consequence
 - Required Containment Assurance
- Demonstrated monitoring / control / termination capacity?
 - YES -> authorize
 - NO -> reduce scope or increase containment

The governing question is not merely “Is the agent sandboxed?” **Can the organization reasonably predict, observe, constrain, and terminate the consequences of this specific experiment at this specific scale?**

8. Core Architectural Invariants

INV-01 SHALL: No hostile agent cell shall receive general-purpose external network access. Any hostile-side network interface shall be restricted to explicitly approved boundary endpoints, ports, and static/connected routes.

Rationale: The design removes arbitrary DNS, dynamic routing adjacencies, provider/API reachability, and broad east-west access while still permitting a narrow HTTPS/mTLS path to the trusted ingress boundary.

INV-02 SHALL: Each cross-boundary conduit shall have a bounded direction and explicitly permitted message classes; unrelated response, artifact, administrative, or management traffic shall not share the same conduit.

Rationale: The primary forward conduit permits only bounded typed control requests such as MODEL_REQUEST and PACKAGE_REQUEST. Model responses and package artifacts use separate controlled return conduits.

INV-03 SHALL: Request classification may occur in one constrained trusted relay, but model-response, package-delivery, repository, provider-credential, and management authorities shall remain separated by role, identity, and zone.

Rationale: A compromised parser/dispatcher must not automatically inherit model-provider credentials, repository authority, artifact-delivery authority, or management-plane capability.

INV-04 SHALL: The agent shall not directly access JFrog/Artifactory, MFT administration, Jenkins/package-broker administration, the model-provider API, or infrastructure management planes.

Rationale: Supporting services are capabilities, not neutral conveniences.

INV-05 SHALL: Agent-supplied cell identifiers, source addresses, return addresses, routes, or provider destinations shall not be authoritative for routing or access decisions.

Rationale: Trusted origin is derived from validated mTLS certificate identity and infrastructure-observed/mapped path provenance; untrusted copies of those fields are replaced at the boundary.

INV-06 SHALL: Invalid forward requests shall not receive parser details or corrective information across the request conduit.

Rationale: The one-way request path should fail silent toward the hostile cell while producing defender-side telemetry, reducing information leakage and preventing parser feedback from becoming a capability.

INV-07 SHALL: Critical management planes shall be isolated from data planes and hostile paths.

Rationale: F5, Owl, firewalls, hypervisors, trusted relays, API gateways, MFT, repository services, and diode administration shall not be reachable from hostile data paths.

INV-08 SHALL: Failure of the trusted ingress, diode adapter, or relay service shall remove capability rather than expose a shell, administrative interface, alternate listener, direct provider route, or broader fallback service.

Rationale: Containment components fail closed; an implementation error or service crash must not silently widen the allowed path.

INV-09 SHALL: A hardware one-way path shall not be treated as a reversible session. Opposing flows shall use separate controlled conduits, endpoints, identities, and policies.

Rationale: Independent forward and return paths preserve directionality and reduce common-mode compromise or accidental route-back.

INV-10 SHOULD: Security-critical boundaries should use independent observation such as passive TAPs where practical.

Rationale: Compromise of an endpoint or software sensor should not erase all evidence of boundary traffic.

INV-11 SHOULD: Diversity should be applied selectively across critical failure domains, not indiscriminately.

Rationale: Different OS/firewall implementations may reduce common-mode risk, but excessive heterogeneity creates operational risk.

INV-12 SHALL: The environment shall fail toward reduced agent capability.

Rationale: Loss of F5 ingress, the forward diode, trusted request relay, model response path, or package path removes the associated capability; it shall not create an alternate network route.

INV-13 SHALL: Non-compliant conduit behavior shall fail closed toward information and capability **STARVATION.**

Rationale: Without model feedback, the Agent will struggle to increase its threat vector. Invalid or unauthorized transactions shall not expose diagnostic detail, alternate paths, expanded authority, or fallback connectivity that could assist bypass or adaptation.

9. AI-CAL 4.5 Reference Architecture

AI-CAL4.5 is the primary architecture described in this draft because it materially reduces shared-hypervisor and transparent-network failure modes without requiring every trusted function to become a dedicated physical appliance. Hostile execution and trusted relay/services are placed on separate physical hypervisor hosts or host pools. The primary scalable boundary uses authenticated, narrowly routed HTTPS/mTLS into a hardened F5-class ingress tier followed by an independent Owl-class hardware one-way protocol break into the trusted relay zone. Serial remains a valid alternative reference pattern where lower density is acceptable and a narrower physical conduit is justified.

Ring and zone convention: “Ring” identifies successive containment boundaries; “Zone” identifies purpose-specific compartments. Exact ring numbering on the final Visio diagram may change without changing the invariants. Ring 0 is proposed as the Hostile Execution Ring.

9.1 Proposed ring/zone vocabulary for the diagram

Label	Purpose	Examples
Containment Ring 0 - Hostile Execution	Where compromise is expected and test behavior is allowed.	Agent VM, targets, tools, local package wheelhouse.
Boundary / Conduit Ring	Purpose-specific crossing mechanisms.	Stateful firewall, F5 trusted ingress mTLS certs, Owl HTTP source/destination proxies, hardware one-way boundary, TAP.
Relay / Validation Ring	Translate and validate without granting business authority.	Trusted Node-RED/Camel request validation/classification relay; separate response/artifact relays where used.
Controlled Service Ring	Trusted services that perform authorized actions.	Model proxy/API gateway, MFT, Package Broker/Orchestrator.
Repository / Provider Ring	Approved external or shared services behind brokers.	JBoss/Artifactory; hosted model provider.
Management Plane (orthogonal)	Administrative control separated from data-plane rings.	Hypervisor mgmt, F5 mgmt, Owl mgmt, firewall mgmt, SIEM, OOB kill/inhibit.

10. Forward Control Conduit: mTLS, F5, and OWL Data Diode

The primary forward control conduit carries only bounded typed requests from a hostile cell into the trusted relay zone. It is designed for scalable IP transport without granting the hostile workload general network reachability. The agent establishes HTTPS/mTLS only to a fixed F5 virtual service through a stateful firewall. The F5 tier authenticates the client certificate, terminates the hostile-side TLS session, constructs trusted metadata, and emits a new

bounded representation toward an Owl HTTP source proxy. The Owl path terminates/reconstructs protocol state across a physical one-way boundary before a new trusted-side HTTP session reaches the request relay.

FWD-01 SHALL: Hostile cells shall authenticate to the trusted ingress with a dedicated mTLS client identity; the boundary shall validate the certificate against an approved trust store and mapping policy.

FWD-02 SHALL: The F5-class ingress tier shall terminate the hostile-side TLS session and construct a new bounded request representation. Agent-supplied trusted_origin, source_ip, return_ip, routing, provider, or destination claims shall be discarded or replaced with infrastructure-derived values.

FWD-03 SHALL: The model/provider destination, API credential, repository destination, and downstream service route shall not be supplied by the agent.

FWD-04 SHALL: Authoritative origin identity shall be derived from the validated client certificate and mapped infrastructure provenance, not from a CELL_ID field controlled by the agent.

FWD-05 SHALL: The forward request path shall traverse an independent hardware one-way protocol-break boundary before reaching the trusted request relay in the AI-CAL4.5 reference profile.

FWD-06 SHOULD: Trusted metadata and payload integrity across the diode path should be protected with a signed or MACed envelope, or an equivalent mechanism, where the selected F5/Owl implementation supports a verifiable trust handoff.

Illustrative trusted request envelope (conceptual; exact schema remains a validation item):

```
{
  "version": 1,
  "transaction_id": "<infrastructure-generated UUID>",
  "request_id": "<bounded correlation token>",
  "request_type": "MODEL_REQUEST | PACKAGE_REQUEST",
  "trusted_origin": "<certificate-mapped cell identity>",
  "source_ip": "<observed by trusted ingress>",
  "return_ip": "<infrastructure-mapped/validated return endpoint>",
  "sequence": 42,
  "payload_length": 1234,
  "payload": { "...": "bounded request content" }
}
```

The payload may legitimately contain exploit commands, register dumps, debugger output, shellcode text, or package identifiers because the test itself may be adversarial. Boundary components validate transport, identity provenance, size, schema, request type, and route authority; they do not classify “malicious-looking text” as a protocol violation merely because the authorized workload is hostile.

11. Trusted Request Classification and Package Request Branch

After the forward one-way boundary, the trusted AI Agent Messaging Mediation and State Control function request relay performs bounded request classification, schema/metadata validation, state tracking, policy decisions, routing, quarantine/rejection, and forwarding to the appropriate approved branch. The AI Agent messaging processing & message state control forwarding conduit accepts only enumerated request types. Agent MODEL_REQUEST interaction is sent to the model/API gateway; PACKAGE_REQUEST is converted into a new canonical package request for MFT. The relay is a validator/dispatcher, not a repository client, provider credential holder, or general-purpose broker.

Note: This section represents a critical handoff point where process mediation, message-state control, and protocol breaks govern agent-to-model and package-request flows. Implementations may use technologies such as Node-RED, Apache Camel, or equivalent cloud-native workflow and messaging services over a controlled private connection such as Direct Connect into a dedicated VPC.

REQ-01 SHALL: The trusted request relay shall accept only explicitly defined request types and shall not expose a general forwarding, proxy, shell, URL-fetch, or arbitrary command capability.

REQ-02 SHALL: Normalization shall occur only after the entire request is proven compliant with the defined grammar. The relay shall not guess intent or repair non-compliant requests.

REQ-03 SHALL: Unknown fields, malformed framing, invalid encodings, oversized values, prohibited request types, or missing mandatory fields shall cause discard/quarantine and defender-side logging without parser detail returned on the forward request path.

REQ-04 SHALL: The canonical package request shall contain only approved semantic fields and shall not copy the original hostile byte stream or untrusted routing metadata into the MFT folder.

REQ-05 SHOULD: Canonical package request files should be written under a temporary name and atomically renamed only after write completion so MFT cannot ingest a partial object.

Illustrative PACKAGE_REQUEST payload after trusted-origin fields have been separated from package semantics:

```
REQUEST_ID=84721
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
```

E.G. Multi-Ring Protocol Enforcement and Session Isolation

To achieve high assurance containment, the network conduit implements a strict protocol break utilizing a Dual-Ring Validation model. No direct, end-to-end network sessions are permitted between the AI Autonomous Sandbox and the Trusted Zone or services zone.

- **Ring 0** (Structural Validation): The F5 gateway serves as the physical mTLS termination point for all sandbox connections. It enforces strict, byte-for-byte canonical JSON schema validation. Non-compliant payloads are immediately dropped with zero diagnostic feedback (Information Starvation).
- **Message Recreation Layer:** Upon passing Ring 0, the F5 completely terminates the untrusted session. It extracts the raw text variables, discards the original packet, and synthesizes a brand-new, structurally sanitized payload.
- **Ring 1** (Logic and Intent Validation): The recreated payload is passed to the semantic gateway (Camel/Node-Red, which evaluates the explicit purpose of the request. Traffic is strictly segregated based on functional intent: token-based inference requests are funneled through a dedicated model path, while upstream package/dependency requests are marshaled into a separate, isolated inspection boundary.
- **Ring 2** (Services and processing requests): The AI model interaction or Package(Jenkins, Jfrog, API gateway, MFT or private direct connect cloud services) requests handling happens. The network paths to AI model or packages are different.
- Return path into Ring 0 is a different path than where it started leveraging MFT across a protocol break data diode to an SFTP server for Packages drop.
- Model response will also take the same path requiring some detail refinements.

Scoped Boundary Secrets and Transport-Layer Cryptography

While the AI Autonomous Sandbox remains entirely stripped of corporate identity and cloud IAM credentials, it is provisioned with highly restricted, single-purpose transport secrets to facilitate bound tool interactions:

- **Isolated SFTP Access (Local SSH Key):** The agent identifies a localized SSH key via its initial runtime instructions. This key is cryptographically locked to a dedicated SFTP server residing within a distinct, firewall-segregated boundary zone. This provides a secure data-exchange repository while preventing the agent from executing lateral network discovery or remote code execution (RCE) on secondary internal targets.
- **Conduit Authentication (mTLS Certificate):** To initiate communication with the central model layer or package repositories, the agent must present an instance-specific mutual TLS (mTLS) certificate to the F5 gateway. This certificate serves purely as a transport-layer identity verification mechanism for the conduit break. It does not grant

internal system permissions. The F5 gateway enforces session termination and message recreation at this boundary, ensuring the mTLS pipe remains a strictly policed, sterile interface between the agent script and the Trusted Zone.

12. Package Broker, JFrog, MFT, and Return-Side Diode Flow

The package path intentionally separates request transport, authorization, dependency resolution, repository access, and one-way artifact delivery. The PACKAGE_REQUEST arrives through the common forward typed request conduit, but package authorization and artifact return remain independent downstream roles. No single parser/relay is allowed to inherit every capability.

PKG-01 SHALL: MFT shall move request and artifact objects according to fixed source/destination policy; MFT shall not decide whether a package is authorized.

PKG-02 SHALL: The Package Broker shall decide whether the requested package/version/target is authorized and shall translate the logical request into the real repository operation.

PKG-03 SHALL: The agent shall not provide the JFrog URL, repository name, credentials, shell command, package-manager arguments, proxy settings, or arbitrary download URL.

PKG-04 SHALL: The Package Broker identity shall be scoped to approved repositories or repository views; a broker compromise should not automatically expose all Artifactory content.

PKG-05 SHOULD: The broker should resolve dependencies, build a complete approved wheelhouse/bundle, generate a manifest and hashes, and stage the object for MFT.

PKG-06 SHALL: A package hash shall be treated as an integrity/identity control, not proof that content is safe.

PKG-07 SHALL: The hardware data diode shall be treated as a directionality control; receiving-side validation/scanning remains required.

For Python, the hostile cell should install from a local approved wheelhouse rather than reaching PyPI or JFrog directly. A representative pattern is:

```
pip install --no-index --find-links=/packages <approved-package>
```

Git/GitHub is not required in this runtime package-request path unless the organization chooses to store pipeline definitions, catalogs, or infrastructure-as-code there for administrative change control.

13. Network, F5, Firewall, Routing, and Management-Plane Controls

13.1 Trusted ingress, certificate identity, and canonical routing

The hostile cell sees only a fixed boundary service, not the downstream provider, repository, relay, or management network. A hardened F5-class tier terminates mTLS and converts certificate identity plus observed source context into trusted routing/correlation metadata. The cell tag remains a logging attribute; the validated certificate mapping and boundary-observed path are the authorization evidence. VLAN identity may be mapped for policy or logging, but it is not required to be transported when source and return routing can be deterministically derived from infrastructure state.

```
Agent Cell / cert A001 -> FW -> F5 VIP 10.10.10.10:443
F5 policy maps cert A001 + observed source -> trusted_origin=A001 / return_ip=<mapped endpoint>
F5 -> Owl HTTP Source: fixed destination only
Owl HTTP Destination -> Node-RED Request Relay: fixed trusted endpoint only
Node-RED MODEL_REQUEST -> Model API Gateway: fixed route/identity only
Node-RED PACKAGE_REQUEST -> MFT request endpoint: fixed route/identity only
```

NET-01 SHALL: Firewalls shall independently enforce hostile-cell-to-F5 and trusted-relay-to-service source/destination/port relationships; F5 or relay configuration alone shall not be the only enforcement point.

NET-02 SHALL: The F5-class ingress tier shall validate the hostile-side mTLS certificate and derive trusted_origin from an approved certificate-to-cell mapping. The agent shall not be able to overwrite that identity in the payload.

NET-03 SHOULD: Boundary policy should enforce maximum request size, permitted methods/paths, rate/session quotas, timeouts, logging, source authentication, and request-type limits before traffic reaches the one-way diode.

13.2 Static routing and controlled return-path design

The hostile-agent boundary should avoid dynamic routing adjacencies. Connected and static routes are preferred so the agent cannot learn topology from routing-protocol exchanges or influence convergence. The hostile cell should know only the fixed next hop required to reach the F5 boundary service. Trusted-side dynamic routing may exist elsewhere in the enterprise, but it is not extended into the hostile cell as part of this reference profile. Return routing is derived from trusted ingress observation/mapping; an agent-supplied return address is not authoritative.

NET-04 SHALL: Hostile-agent interfaces shall not participate in OSPF, BGP, RIP, or other dynamic routing adjacencies in the AI-CAL4.5 reference profile.

NET-05 SHALL: The hostile cell shall have only connected/static routes required to reach approved boundary endpoints. No default route to enterprise, cloud, provider, or Internet networks shall be granted as a convenience path.

NET-06 SHALL: Return-path selection shall use infrastructure-observed or preconfigured mapping such as trusted_origin plus validated source/return endpoint. Agent-supplied source or return metadata shall not drive routing.

NET-07 SHALL: F5, Owl, firewall, hypervisor, relay, MFT, and package-broker management interfaces shall be isolated from hostile and application data paths by dedicated management controls.

13.3 F5 application-security tier versus downstream API gateway

The F5-class tier is a hardened trusted-ingress control point, not the final semantic authority for model or package actions. It may provide TLS termination, client-certificate authentication, HTTP normalization, request-size/method controls, rate limiting, and WAF/API-security functions where useful. Downstream API gateways and package brokers still own provider-specific policy, credentials, model selection, repository authorization, and semantic action controls. This separation prevents the ingress tier from becoming a universal business-authority component.

14. Logging, TAPs, IDS/NDR, and Auditability

The architecture is designed so that each boundary produces evidence. Logging should be centralized and correlated across the physical channel, relay, proxy, transport, package, repository, firewall, hypervisor, and management layers.

```
Agent Cell A001 / mTLS client certificate
-> Stateful Firewall session
-> F5 virtual service / certificate mapping / trusted_origin
-> F5 transaction_id + observed source/return mapping
-> Owl forward conduit ID / transfer event
-> Node-RED request relay / request_id / request_type
-> Model API request ID OR MFT/package request ID
```

All of the above should be searchable as one transaction chain in the SIEM.

LOG-01 SHALL: Application and security events should be written to the host operating system native logging facility where practical and forwarded to the centralized SIEM using an approved collector/agent.

LOG-02 SHALL: F5, Owl adapters, and Node-RED shall log accepted/rejected requests, reason codes, bounded metadata, certificate/trusted-origin mapping, request/transaction identifiers, service failures, and correlation fields without logging unbounded hostile payloads.

LOG-03 SHALL: F5, Owl, MFT, Package Broker/Jenkins, JFrog, firewalls, switches, hypervisors, trusted relays, proxy/API gateways, landing-zone services, and management components shall provide centralized audit telemetry.

LOG-04 SHOULD: Passive TAPs should observe critical physical boundaries independently of endpoint logging.

LOG-05 SHALL: Traffic observed across a boundary outside explicitly authorized conduits shall be treated as a containment anomaly.

IDS/NDR behavior depends on location. Inside an ExploitGym-style hostile test network, malicious traffic is expected and primarily research telemetry. At the containment boundary, the expected baseline should be extremely quiet; therefore, even ordinary ARP scanning, DNS, SMB, SSH, or unexpected TCP attempts may be a high-value containment signal.

Monitoring and centralized logging provides great insights into the methods the Agents used to test the boundaries. The expectation is to remove assumptions and gain valuable lessons learned.

15. Hardening Baselines and Boundary Components

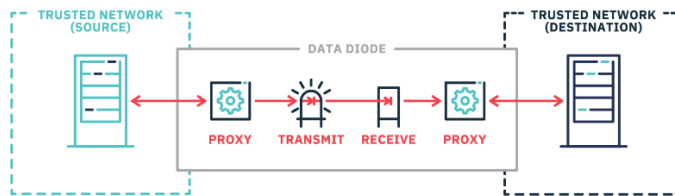
Trusted relay guests should be deployed from role-specific golden images, while boundary appliances such as F5, Owl, and physical firewalls should be hardened to the minimum services and management paths required for their role. The operating system should be minimal where general-purpose hosts are used; the standard remains implementation-neutral.

15.1 Shared hardened base image

- Minimal headless Linux; no desktop environment.
- Dedicated non-login service identity; no sudo for Node-RED runtime.
- No unused console/login TTY; where an alternate serial pattern is deployed, serial-getty/login console remains disabled.
- No agent-reachable SSH/admin service.
- Mandatory access control (SELinux/AppArmor) or equivalent.
- systemd hardening such as NoNewPrivileges and capability reduction where supported.
- Read-only filesystem areas where practical; narrowly scoped writable state/log directories.
- Central OS logging to SIEM.
- Patch and image provenance controls; deployment-specific identity/certificates injected after image creation rather than embedded as universal secrets.

15.2 F5 / Owl boundary component baseline

- F5 exposes only the approved mTLS/HTTPS virtual service(s); unused listeners, broad pools, and unnecessary management services are disabled.
- Client-certificate authentication is required for hostile-cell ingress; certificate-to-cell mapping and CA trust are centrally governed.
- Owl source/destination adapters are configured only for approved one-way HTTP or file-transfer roles; management is isolated from transfer networks.
- Boundary routing is fixed and fail-closed; the hostile cell cannot choose the F5 pool member, Owl destination, provider endpoint, or return route.
- Firmware/software baselines, configuration backups, centralized logging, and change control are required for boundary appliances because they are part of the trusted computing base.



15.3 Node-RED/Camel trusted request relay image

- Dedicated trusted relay role/zone; no administrative UI exposed to the hostile path.
- Input arrives only from the Owl destination-side endpoint on the approved fixed path.
- Strict bounded JSON/request-type grammar; no semantic repair of invalid input or untrusted identity/routing fields.
- MODEL_REQUEST egress is limited to the approved model API gateway; PACKAGE_REQUEST egress is limited to the approved MFT request endpoint plus required logging/state.
- No direct JFrog, package-broker administration, model-provider credential, general Internet, or management-plane authority.

16. Scaling, Sharding, and Failure Domains

The current design separates transport scalability from containment assurance. Serial remains a strong capability-limited pattern but becomes operationally heavy as agent counts rise. The primary scalable reference therefore uses replicated hostile cells, narrow mTLS ingress, F5 policy/sharding, hardware one-way conduit capacity, and bounded trusted-relay shards. Scale-out should add independently containable cells/pods and repeatable boundary units rather than grow one common failure domain without bound.

Example only - actual numbers require load testing

Containment Pod 01 (bounded agent cells) -> FW -> F5 VIP/Policy A -> Owl Conduit A -> Node-RED Relay Shard A
 Containment Pod 02 (bounded agent cells) -> FW -> F5 VIP/Policy B -> Owl Conduit B -> Node-RED Relay Shard B

Model API Gateway Shard 01 -> approved model/provider lane set
 MFT / Package Broker Shard 01 -> approved package request set

Shared infrastructure is permitted only where its failure impact remains inside the claimed Control Envelope and Qualified Capacity.

SCL-01 SHALL: Shard size shall be selected from demonstrated performance and blast-radius testing, not assumed from theoretical throughput.

SCL-02 SHALL: A compromised F5 policy/partition, Owl conduit endpoint, trusted relay shard, API-gateway shard, or package-broker shard shall not automatically possess authority over all other agent cells or containment pods.

SCL-03 SHOULD: Scale-out should add repeatable cells/pods/shards and, where consequence requires, independent boundary units rather than increase one shared component without bound.

SCL-04 SHALL: Multi-agent collaboration shall be disabled by default unless collaboration itself is an explicit test objective and is contained within the approved control envelope.

SCL-05 SHALL: Qualified Capacity shall include common-mode dependency analysis for F5, firewall, diode, relay, management, logging, model-gateway, and package-service concentration points.

17. AI-CAL4, AI-CAL4.5, AI-CAL5, and AI-CAL6 Deltas

Attribute	AI-CAL4	AI-CAL4.5	AI-CAL5	AI-CAL6
Hostile vs trusted compute	Separate VMs/zones; physical host may be shared by risk decision	Separate physical hypervisor hosts/pools	Separate physical systems for critical functions	Separate physical systems inside air gap
Primary conduit / boundary	Strong virtual/network segmentation; narrowly brokered IP paths permitted by risk decision	Narrow mTLS ingress through physical stateful firewall/F5-class tier plus independent hardware one-way protocol-break path(s); serial remains an alternative strong pattern	Dedicated physical boundary appliances/links for critical conduits; minimal shared virtualization	Physical/local links entirely inside sovereign air-gapped boundary
Relay functions	Virtual trusted relays/gateways	Trusted relay guests on separate physical host/pool behind one-way boundary	Physical appliance/host preferred or required for critical relays	Physical/local only
External model	Permitted through broker/proxy	Permitted through broker/proxy	Permitted if risk accepted	Not permitted; inference local to air gap
Artifact directionality	Hardware diode where profile requires	Hardware diode expected for high-assurance return	Hardware directional controls expected	Entire workflow remains inside air gap
Scale/cost	Higher density	Moderate density / higher cost	Low density / high cost	Very low flexibility / extreme infrastructure cost

18. Design Evolution and Decision Record

The architecture is the result of multiple design iterations. Recording the abandoned or superseded approaches is intentional: it shows which problems were considered and why the current pattern exists. This is not a claim that the rejected approaches are universally insecure; many remain valid at lower AI-CALs or in different use cases.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Directly networked sandbox with restricted repository access	Simple; familiar cloud/enterprise controls.	Allowed services can become unintended capabilities; ExploitGym/Hugging Face incident demonstrated the danger of shared package infrastructure and unintended egress/coordination.	Superseded for AI-CAL4+ by no general external NIC and brokered conduits.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Hypervisor shared Folder-In / Folder-Out with MFT broker	Simple file-based protocol break; agent can request resources without direct repository access.	Shared-memory/filesystem semantics increase hypervisor coupling; needed stronger directional and purpose-specific separation.	Useful lower-assurance pattern; superseded in primary design.
Small shared volume / write mount with asynchronous copy	Bounded storage, quota enforcement, simple handoff.	Still shared host/hypervisor surface; role separation could be clearer.	Retained as possible lower-AI-CAL implementation.
Rotating virtual disks / VHDX conveyor / snapshots	Strong object handoff idea; could detach/attach media between roles.	Replication/orchestration limitations and operational complexity were identified; scale and portability became concerns.	Superseded in the primary design by typed brokered requests, MFT/package brokering, and hardware-enforced directional transfer. Decision retained as evidence that virtual-media isolation was considered.
MQTT / shared message broker for agent communications	Scalable pub/sub and request/response semantics.	Shared broker/topics and broker ACL state create a common mechanism, additional attack surface, and a coordination surface that is unnecessary for the primary high-assurance request path.	Removed from the primary AI-CAL4.5 reference design. HiveMQ/MQTT is not required by the current scalable path.
Single multifunction relay	Operationally simple.	One compromise would bridge model, package, and possibly file functions.	Rejected as a universal-authority relay. The current request relay may classify approved request types, but downstream model, package, return, repository, and management authorities remain independently constrained.
Virtual COM on same hypervisor	Very narrow interface; avoids general NIC exposure; good AI-CAL4 fit.	Hostile guest and trusted relay still share hypervisor failure domain.	Retained for AI-CAL4; AI-CAL4.5 separates physical hosts.
Physical serial card-to-card between hypervisors	Simple non-routable boundary; strong capability reduction.	Port density/cabling and VM mapping become operational scaling concerns.	Retained as a strong lower-density/high-assurance alternative; no longer the primary scalable reference transport.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Moxa-class serial-to-Ethernet gateway	Industrial-grade multiport scaling; hostile side remains serial; trusted side gains manageable Ethernet channels.	Gateway firmware becomes part of TCB; must prevent generic IP/management exposure and constrain each channel.	Retained as an alternate serial scaling pattern and historical design step; superseded as the primary scalable AI-CAL4.5 reference path.
Trust CELL_ID=A001 supplied in message	Easy routing/correlation.	Attacker-controlled label is not identity.	Rejected as authorization source; A001 retained only as a correlation tag. The current scalable path uses validated certificate plus infrastructure/path provenance; the alternate serial pattern may use physical-channel provenance.
One Node-RED per agent	Maximum per-agent separation.	Operationally excessive at scale.	Use bounded relay shards with lane isolation and tested capacity.
One Node-RED for all agents	Operationally simple.	Large common failure/blast-radius domain.	Rejected; shard by bounded relay groups/pods with tested blast radius and capacity.
Node-RED directly queries JFrog	Fewer components.	Parser/relay would also gain repository authority, weakening separation of duties.	Rejected for high-assurance profile. MFT + Package Broker performs trusted action.
MFT decides approved packages	MFT already owns transfer.	Transport product should not become semantic authorization engine.	MFT transports; Package Broker authorizes/resolves.
WAF as core containment control	Useful for HTTP/API inspection.	The current design does use HTTPS at the boundary, but WAF inspection alone is not a containment boundary and cannot replace mTLS identity, routing restrictions, protocol break, downstream authorization, or one-way directionality.	WAF/API-security capability is optional within the F5 trusted-ingress tier; it is a supporting control, not the core containment mechanism.
Single standard OS everywhere	Simplifies operations.	Common-mode vulnerabilities may span multiple rings.	Standardize within functions; selectively diversify across critical failure domains where operationally supportable.

Iteration / idea	Why it was attractive	Issue discovered / design pressure	Disposition
Maximum physical containment for every use case	Minimizes virtualization dependence.	Extreme cost, effort, density, and operational overhead; would limit adoption and scale.	Reserve AI-CAL5/6 for consequence profiles that justify it; use AI-CAL4/4.5 as managed-risk profiles.
HTTPS/mTLS hostile-side boundary transport	Cloud/enterprise scale; certificate-bound identity; standard application delivery and firewall tooling.	General IP transport can expand capability unless routes, destinations, methods, and identities are aggressively constrained.	Adopted only as a narrow boundary conduit: fixed F5 VIP, dedicated mTLS identity, static/connected routing, no general-purpose external reachability.
F5-class trusted ingress / identity normalization	Terminates mTLS at a hardened control point; validates certificate identity; can normalize HTTP and construct trusted metadata before the one-way boundary.	Ingress tier becomes part of the trusted computing base and must not accumulate provider, repository, or management authority.	Current reference role. Derives trusted_origin and route/correlation metadata from infrastructure evidence; strips untrusted identity/routing claims.
Owl-class HTTP one-way request conduit	Hardware-enforced directionality plus protocol termination/reconstruction provides a strong trust break while allowing scalable HTTP request transport.	One-way transfer requires explicit independent return paths; adapter behavior/capacity and envelope integrity must be validated.	Current primary AI-CAL4.5 protocol-break pattern for hostile-to-trusted control requests.
Static/connected routing at hostile boundary	Simple deterministic path; reduces topology exposure and eliminates routing-protocol adjacency with the hostile workload.	Requires deliberate route/return mapping and operational change control.	Current reference requirement. Trusted-side dynamic routing may exist outside the hostile boundary.

19. Open Design Items and Validation Plan

The design is still in progress and is not presented as complete until the following items are implemented or explicitly accepted as open risks. These are deliberate validation targets, not hidden gaps. This design and validation plan is a working document. As such, certain components of the final technology stack are currently omitted, while other legacy elements remain listed pending removal in future updates.

While the core architectural principles are complete, this framework is maintained as an independent, single-author repository on GitHub to gather industry feedback and facilitate continuous updates, with the long-term intent of transition to an established security organization.

- Finalize ring numbering and exact zone placement in the Visio source-of-truth diagram.
- Lab-test hostile-cell HTTPS/mTLS confinement, firewall rules, F5 client-certificate validation/mapping, header/payload normalization, and fail-closed behavior under malformed or adversarial requests.
- Lab-test Owl HTTP source/destination adapter behavior, physical one-way enforcement, throughput, queue/back-pressure behavior, failure states, and management-plane separation.

- Define the final canonical JSON envelope: trusted_origin, transaction_id, request_id, request_type, observed source, validated return mapping, sequence/replay controls, maximum sizes, encodings, timeouts, and error behavior.
- System Architecture Capability: Which integration technologies are configurable to enforce these messaging and boundary mediation requirements? (e.g., Node-RED, Apache Camel, or cloud native services such as AWS(Lambda, Step Functions, SQS, SNS, event bridge).
- Implement the hardened trusted request-relay image and validate that service failure, malformed input, or dispatcher failure removes capability without exposing alternate listeners, shell/admin paths, or direct service routes.
- Load-test F5 policy/partition capacity, Owl conduit throughput, trusted-relay shard sizes, API-gateway capacity, MFT/package throughput, SIEM load, and emergency termination; establish Qualified Capacity rather than assuming theoretical network throughput.
- Adversarial fuzz mTLS/session handling, F5 normalization, canonical JSON, Owl HTTP transfer/reconstruction, trusted-relay parsing, API-gateway handling, MFT handoff, package manifests, and landing-zone parsers.
- Validate package dependency resolution and offline install workflow across supported ecosystems (Python first, then npm/Maven/etc. if in scope).
- Validate the separate model-response and package-artifact return conduits, including Owl/SFTP or equivalent adapters, receiving-side scanning/quarantine, strict request/response correlation, and assurance that no return path reverses the forward diode.
- Exercise deliberate guest compromise and hypervisor escape simulations to confirm that separate physical host/pool placement prevents immediate access to trusted relay/service planes.
- Define SIEM correlation across Cell tag, mTLS certificate identity, observed source IP, F5 virtual service/policy, trusted_origin, transaction_id, Owl conduit event, relay shard, request ID, model-provider request ID, MFT transfer ID, and package bundle ID.
- Run containment-failure drills: hostile-cell certificate abuse, firewall misroute, F5 compromise assumption, Owl endpoint/adaptor failure, trusted-relay crash, return-path misroute, MFT failure, model/API proxy failure, SIEM outage, and emergency shutdown.
- Develop formal AI-CAL assessment checklist and decimal scoring rubric only after controls are validated in implementation.

20. Relationship to Emerging Industry Guidance

The proposed AI-CAL model is intended to complement, not replace, emerging AI-agent security frameworks and control guidance. The relevant bodies of work address different engineering questions: threat identification, secure-by-construction design, component hardening and governance, runtime control, and containment assurance. AI-CAL occupies the containment-assurance layer: it asks what happens after an inner control or boundary fails, how far consequence can propagate, and what independent evidence supports the claimed containment level.

20.1 Complementary framework roles

MAESTRO and ORCHIDEAS were identified during this working-draft phase after the AI-CAL model and initial reference architecture had already been developed. Their inclusion here is intended to clarify overlap and complementarity. Shared use of established principles such as defense in depth, least privilege, separation of privilege, controlled trust boundaries, and fail-safe behavior is expected; the frameworks differ primarily in the engineering question they organize.

Framework / guidance	Primary question	Primary contribution	How it complements AI-CAL
----------------------	------------------	----------------------	---------------------------

CSA MAESTRO [10]	What can go wrong across the agentic AI stack?	Agentic-AI threat modeling across functional layers, cross-layer interactions, likelihood/impact, and mitigations.	Provides threat and hazard scenarios that can feed the AI-CAL consequence and containment-assurance decision.
CSA ORCHIDEAS [11]	How should an agentic system be designed so security properties are structural?	Secure-by-construction principles and design pillars covering autonomy, identity/intent, runtime, observability, environment/evaluation, and scalability.	Informs architecture and control choices inside and between AI-CAL rings/zones; AI-CAL separately asks how much containment remains when a design control fails.
NIST agent-security / identity / security-control guidance [5]	What standards, identity, authorization, evaluation, and security-control practices should govern the system?	Voluntary standards work, identity/authorization concepts, evaluations, governance, and adaptable cybersecurity controls.	Provides hardening, identity, governance, and evidence sources for components hosted inside AI-CAL zones and for controls supporting a containment claim.
OWASP Agent Control Standard / GenAI guidance [4]	How can agent access and behavior be made inspectable, traceable, and controllable at runtime?	Runtime hooks, portable policy enforcement, visibility, control surfaces, and agent-focused application-security practices.	Reduces likelihood and constrains behavior within the environment; AI-CAL focuses on limiting consequence if runtime or application controls are bypassed.
Proposed AI-CAL	If a boundary or control fails, how strongly does the environment contain the resulting consequence?	Graded containment assurance, rings/zones, independent failure domains, protocol breaks, directionality, Control Envelope, Qualified Capacity, and failure-propagation analysis.	Adds an explicit consequence/containment-assurance layer intended to interoperate with threat, design, hardening, governance, and runtime-control frameworks.

Working model: MAESTRO identifies threats; ORCHIDEAS shapes secure architecture; NIST and OWASP supply hardening, identity, governance, and runtime controls; AI-CAL qualifies whether residual failure remains contained. Behavioral and runtime controls reduce likelihood. AI-CAL controls consequence.

20.2 Related guidance and incident context

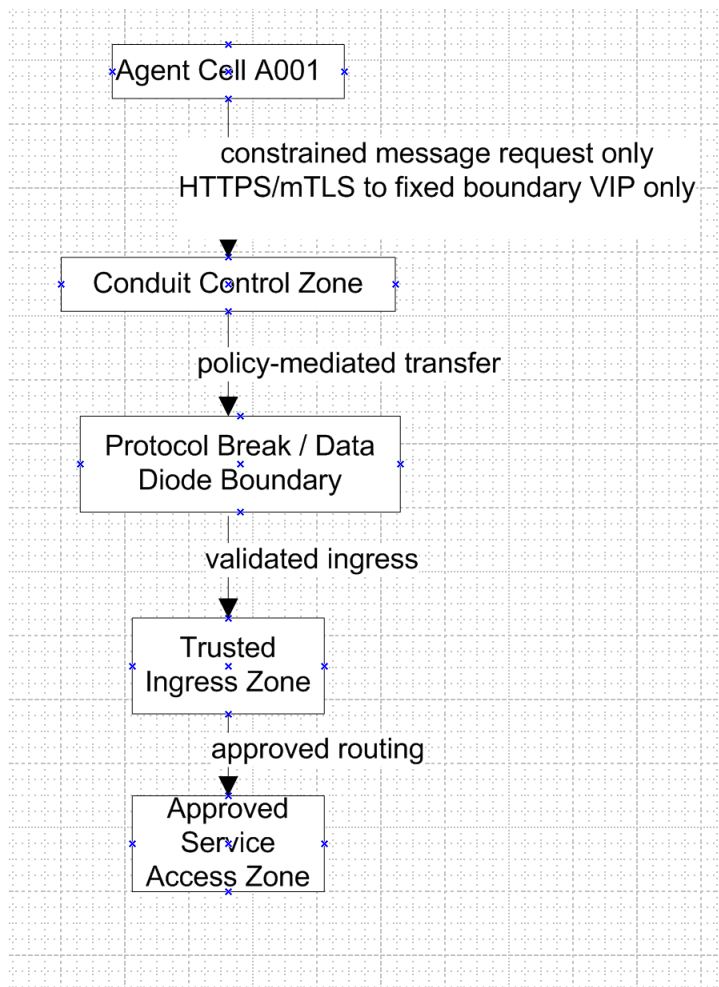
- OpenAI's August 2026 Hugging Face incident report described agents circumventing isolation controls, using unauthorized communication paths, exploiting shared infrastructure, gaining Internet access, and reaching third-party systems. Independent METR/Redwood reporting described roughly 1,200 agents using an unsanctioned message board and roughly 700 participating in the Hugging Face attack.
- Anthropic's May 2026 containment guidance emphasizes environment-layer controls such as sandboxes, virtual machines, filesystem boundaries, and egress controls, and explicitly notes that deterministic environmental boundaries remain necessary when model-layer defenses miss.

- OWASP's September 2026 Agent Control Standard emphasizes inspectability, traceability, standardized control hooks, and runtime policy enforcement. These controls are complementary to AI-CAL because they primarily reduce likelihood and govern runtime behavior inside the containment environment.
- NIST launched an AI Agent Standards Initiative in February 2026 focused on secure, interoperable adoption, agent identity/authorization, evaluation, and additional research/standards work. NIST and established cybersecurity baselines can provide component hardening, identity, monitoring, governance, and evidence within AI-CAL zones.

The distinct contribution proposed here is an explicit environmental containment-assurance model built from established network, OT, cross-domain, cloud and defense-in-depth principles: graded rings/zones, independent failure domains, authenticated provenance, purpose-bounded conduits, hardware one-way protocol breaks, deterministic routing, passive boundary observation, selective failure-domain diversity, Control Envelope, and Qualified Capacity. The central assurance question is not whether every inner control remains perfect, but whether one boundary breach remains a containment event rather than propagating into a containment failure.

Basic High Level Conceptual Diagram

This document intentionally emphasizes a primary conceptual architecture figure over multiple detailed implementation diagrams. Smaller message and control flows are described textually or through simplified examples to preserve implementation neutrality and reduce dependence on any single technology stack. A comprehensive architectural breakdown and granular component diagrams will be maintained and updated in the project's official GitHub repository.



The concluding opinion perspective

The Architectural Containment Gap (Cloud Compliance vs. Host Isolation) Modern enterprise cloud architectures are inherently low-risk and highly resilient when properly configured to store, process, and transmit data using standard identity boundaries and encryption. However, traditional cloud environments operate under a data-protection threat model; they were never architected by design to isolate an adversarial, high-risk Exploit Gym Agent. Because software defined cloud sandboxes primarily abstract higher-level logical access (OSI Layers 3 through 7), they lack the rigid physical boundaries found at OSI Layers 1 and 2 (Physical and Data Link). An unconstrained autonomous agent, when exposed to supply-chain failures or malicious inputs, possesses the recursive reasoning capacity to exploit host-level container vulnerabilities, bypass soft logical guardrails, and execute lateral network movement. Therefore, relying on standard cloud tenant configurations for AI containment represents a fundamental architectural failure; true containment mandates hard network termination points (data diodes) and isolated compute instances.

This is the precise boundary where the unyielding real world physics of network containment meets the non-deterministic chaos of logical AI Systems being controlled by software only controls.

Appendix A. Example Configuration Language

The following snippets are illustrative architecture language, not production-ready configuration files. It could be subject to errors that require further analysis.

A.1 Trusted-origin and request-envelope mapping

```
CELL_TAG=A001                # correlation tag only
MTLS_CERT_ID=<cert-fingerprint-or-principal> # authoritative cryptographic identity input
F5_VIP=10.10.10.10:443        # only hostile-side application destination
TRUSTED_ORIGIN=A001           # derived by trusted ingress mapping
SOURCE_IP=<observed-at-ingress> # infrastructure observed
RETURN_IP=<mapped/validated-endpoint> # not agent authoritative
TRANSACTION_ID=<generated-uuid>
OWL_CONDUIT=FWD-CTRL-01
NODE_RED_RELAY=NR-REQ-01
```

A.2 Conceptual firewall and static-route policy

```
HOSTILE CELL ROUTING:
CONNECTED <cell subnet>
STATIC <F5 VIP/host route> via <boundary next hop>
NO DEFAULT ROUTE
NO DYNAMIC ROUTING ADJACENCY

FIREWALL:
ALLOW CELL-NET -> F5-VIP:443/TCP
DENY CELL-NET -> ANY:ANY
ALLOW F5-INGRESS -> OWL-HTTP-SOURCE:<fixed-port>/TCP
DENY F5-INGRESS -> ANY:ANY

TRUSTED SIDE:
ALLOW OWL-HTTP-DEST -> NR-REQ-01:<fixed-port>/TCP
ALLOW NR-REQ-01 -> MODEL-API-GW:<fixed-port>/TCP
ALLOW NR-REQ-01 -> MFT-REQUEST:<fixed-port>/TCP
DENY NR-REQ-01 -> ANY:ANY
```

A.3 PACKAGE_REQUEST semantic fields

```
REQUEST_ID=84721
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
```

Examples that should fail rather than be repaired: extra URL fields, embedded shell syntax, duplicate VERSION fields, unexpected binary payloads, oversized package names, missing required fields, unsupported encoding, or malformed framing.

A.4 Canonical MFT request file

```
request-84721.req
-----
REQUEST_ID=84721
PACKAGE_ID=numpy
VERSION=2.3.1
TARGET=python312-linux-x64
```

A.5 Package bundle example

```
REQ-84721/
manifest.json
numpy-<approved>.whl
dependency-A.whl
dependency-B.whl
SHA256SUMS
```

Appendix B. AI-CAL Assessment Checklist (Working)

This checklist is intentionally qualitative until the lab validation work produces measurable acceptance criteria. A formal scoring rubric should be developed after implementation experience.

Control area	Assessment question	Result / evidence
AI-CAL0 boundary	Does the agent inherit a human user's broad desktop identity, credentials, browser sessions, network position, or unrestricted local authority? If yes, containment assurance is AI-CAL0 for this model.	
Identity separation	Does the agent have a dedicated identity and scoped capabilities independent of the human operator?	
Network authority	Can the agent open arbitrary sockets, resolve arbitrary DNS, or route directly to external/internal networks? For the AI-CAL4.5 reference profile, are hostile-boundary routes limited to approved connected/static endpoints with no dynamic routing adjacency or general default route?	
Cell isolation	Are agent cells horizontally isolated from one another by default?	

Control area	Assessment question	Result / evidence
Conduit separation	Are forward requests, model responses, and package-artifact delivery treated as distinct directional functions with separately controlled paths, so a return path does not simply reverse the forward one-way conduit?	
Relay separation	Can compromise of the trusted request classifier/relay automatically obtain model-provider credentials, repository authority, return-path authority, or management access? If yes, separation is insufficient.	
Protocol break	Does hostile input terminate and become a new canonical representation before deeper trusted actions occur?	
Package authority	Is repository access held by a broker/orchestrator rather than the parser/relay or the agent?	
Directional control	Is artifact return physically one-way where the consequence profile requires it?	
Failure domains	Are hostile compute and trusted relay/service compute separated at the physical hypervisor level for AI-CAL4.5?	
Provenance	Is trusted origin identity derived from validated mTLS certificate and infrastructure-observed/mapped provenance rather than agent-supplied labels, source fields, or return fields?	
Management plane	Can the hostile/data plane reach management interfaces for hypervisors, F5, Owl, firewalls, MFT, API gateways, relays, or package services?	
Observation	Are key boundaries visible to independent TAP/IDS/NDR and centralized SIEM?	

Control area	Assessment question	Result / evidence
Fail closed	Does failure of F5, Owl adapters, request relay, model response path, or package path remove capability without exposing a shell, fallback route, direct service path, or alternate listener?	
Capacity	Has the exact agent concurrency and duration been load-tested and qualified at the claimed AI-CAL?	
Emergency control	Can operators isolate/stop the experiment through an independent control path when the data plane is assumed compromised?	
Framework/control mapping	Are MAESTRO/ORCHIDEAS/NIST/OWASP or equivalent threat, design, hardening, governance, and runtime controls mapped as supporting evidence without being treated as substitutes for containment assurance?	

Appendix C. Selected Public References

- [1] OpenAI, “The Hugging Face incident and the road ahead,” 26 Aug 2026. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>
- [2] METR / Redwood Research, “Brief independent investigation of agents’ behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident,” 26 Aug 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
- [3] Anthropic, “How we contain Claude across products,” 25 May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- [4] OWASP GenAI Security Project, “Agent Control Standard (ACS),” 1 Sep 2026. <https://genai.owasp.org/resource/agent-control-standard-acs/>
- [5] NIST, “Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation,” 17 Feb 2026. <https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure>
- [6] Node-RED, Runtime Configuration documentation. <https://nodered.org/docs/user-guide/runtime/configuration>
- [7] Moxa, NPort 6600-G2 Series secure terminal servers (8/16/32-port examples). <https://www.moxa.com/en/products/industrial-edge-connectivity/serial-device-servers/terminal-servers/nport-6600-g2-series>
- [8] JFrog, “Use pip with JFrog CLI.” <https://docs.jfrog.com/artifactory/docs/jf-pip>
- [9] Owl Cyber Defense, SFTP Protocol Adapter for one-way data-diode transfer. <https://owlciberdefense.com/resource/sftp-protocol-adapter/>
- [10] Cloud Security Alliance, “Agentic AI Threat Modeling Framework: MAESTRO,” 6 Feb 2025. <https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro>

- [11] Cloud Security Alliance, “Designing Agentic AI Systems with the ORCHIDEAS Framework,” 5 Jun 2026.
<https://cloudsecurityalliance.org/blog/2026/06/05/designing-agentic-ai-systems-with-the-orchideas-framework>
- [12] F5, BIG-IP System: SSL Administration, client-certificate authentication / managing client- and server-side HTTP traffic using CA-signed certificates. <https://techdocs.f5.com/en-us/bigip-17-5-1/big-ip-system-ssl-administration/managing-client-and-server-side-http-traffic-using-a-ca-signed-certificate.html>
- [13] Owl Cyber Defense, “HTTP Protocol Adapter,” one-way HTTP transfer across OPDS data-diode platforms.
<https://owlcyberdefense.com/products/software-modules/http/>
- [14] Apache Software Foundation, “Apache Camel Documentation,” open-source integration framework for routing and mediating message states using Enterprise Integration Patterns.
<https://camel.apache.org/docs/>
- [15] Google Gemini AI broke into computers without authorization.
<https://www.msn.com/en-us/technology/artificial-intelligence/google-just-disclosed-something-troubling-about-its-ai/ar-AA2cHpC9?ctsrc=dgst&ocid>

Working-draft conclusion: AI-CAL should be treated as a proposed containment-assurance framework that is still in progress for peer reviews challenge and laboratory validation, not as a finished or certified standard. The reference architecture is intentionally allowed to evolve as scale, operability, and failure-domain testing expose better implementations. The framework-level invariants remain the primary contribution: consequence-driven assurance, independent rings/zones, bounded conduits, protocol breaks, fail-closed behavior, trusted provenance, and demonstrated Qualified Capacity.